

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



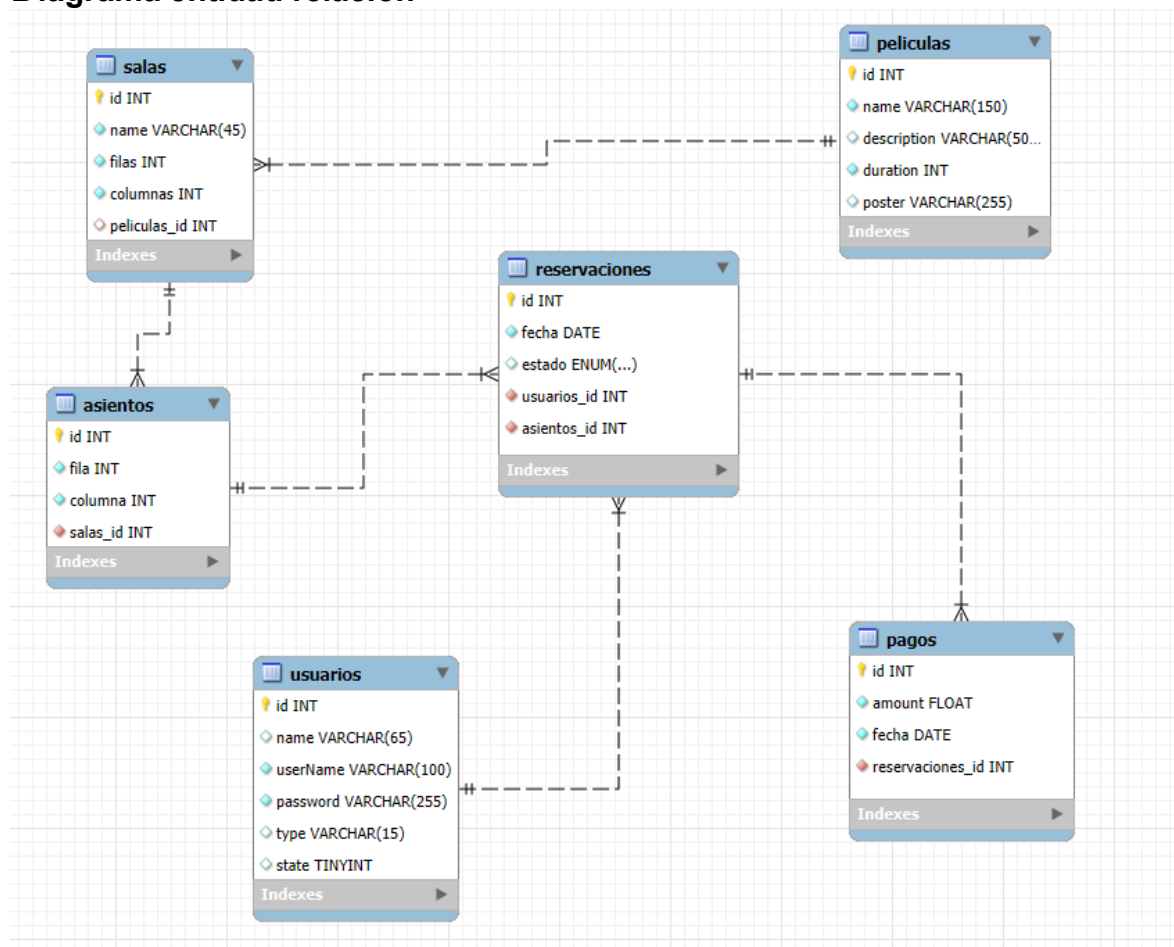
PROYECTO

Primera Entrega, Avances

AUTOR

Sajvin Gómez, Mario Daniel (1612921)

Diagrama entidad relación



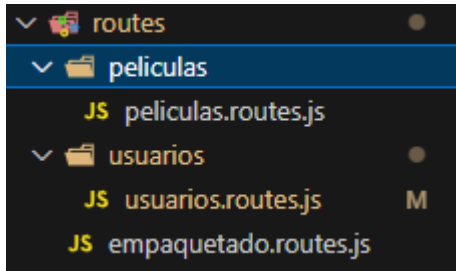
Código

Rutas

Server

```
backend > src > JS server.js > ...
 1  // configuracion del servidor
 2  const express = require("express");
 3  const port = 3000;
 4  //require("dotenv").config();
 5
 6  const app = express();
 7  const empaquetado = require("../routes/empaquetado.routes");
 8
 9  app.use(express.json());
10
11  app.use("/api", empaquetado);
12
13  app.listen(port, () => {
14    | console.log(`Servidor corriendo en http://localhost:${port}`);
15    | });
16
```

Cómo están las rutas



```
backend > src > routes > JS empaquetado.routes.js > ...
1  const express = require("express");
2  const empaquetado = express.Router();
3
4  const peliculaRouter = require("../películas/películas.routes");
5  const usuariosRouter = require("../usuarios/usuarios.routes");
6
7  empaquetado.use("/pelicula", peliculaRouter);
8  empaquetado.use("/usuarios", usuariosRouter);
9
10
11  module.exports = empaquetado;
12
```

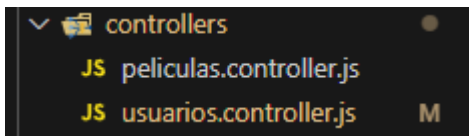
```
backend > src > routes > películas > JS películas.routes.js > ...
2  const peliculaRouter = express.Router();
3
4  const {
5    crearPelicula,
6    buscarPeliculaPorNombre,
7    actualizarPelicula,
8    eliminarPelicula,
9    listarPeliculas,
10 } = require("../controllers/películas.controller");
11
12 peliculaRouter.use((req, res, next) => {
13   console.log("revisar si es admin y esta auth");
14   next();
15 })
16
17 // Definir la ruta
18 peliculaRouter.post("/crearPelicula", crearPelicula);
19 peliculaRouter.get("/buscarPelicula", buscarPeliculaPorNombre);
20 peliculaRouter.put("/actualizarPelicula/:id", actualizarPelicula);
21 peliculaRouter.delete("/eliminarPelicula/:id", eliminarPelicula);
22 peliculaRouter.get("/listarPeliculas", listarPeliculas);
23
24 module.exports = peliculaRouter;
25
```

```

backend > src > routes > usuarios > JS usuarios.routes.js > ...
1  const express = require("express");
2  const usuariosRouter = express.Router();
3
4  const { verificarToken, verificarAdmin } = require("../middlewares/auth");
5
6  const {
7    obtenerUsuarios,
8    registrarUsuario,
9    loginUsuario,
10   cambiarRolUsuario,
11  } = require("../controllers/usuarios.controller");
12
13  // Definir la ruta
14  usuariosRouter.get("/listarUsuarios", verificarToken, verificarAdmin, obtenerUsuarios);
15  usuariosRouter.post("/registrarUsuario", registrarUsuario);
16  usuariosRouter.post("/loginUsuario", loginUsuario);
17  usuariosRouter.put("/cambiarRolUsuario/:userId", verificarToken, verificarAdmin, cambiarRolUsuario);
18
19  module.exports = usuariosRouter;
20

```

Controladores



Peliculas.controller.js

```

const pool = require("../config/db");
const queries = require("../database/peliculasQueries");

const crearPelicula = async (req, res) => {
  const { name, description, duration, poster } = req.body;

  // validar que el nombre sea un string no vacío
  if (typeof name !== "string" || name.trim() === "") {
    return res.status(400).json({
      message:
        "El nombre de la película es obligatorio y debe ser un texto válido.",
    });
  }

  // validar la duración, que sea un entero y mayor a cero
  if (isNaN(duration) || duration <= 0) {
    return res.status(400).json({
      message: "La duración debe ser un número mayor que 0.",
    });
  }
}

```

```

// validar la descripción, que sea un string no vacío
if (typeof description !== "string" || description.trim() === "") {
  return res.status(400).json({
    message: "La descripción es obligatoria y debe ser un texto válido.",
  });
}

try {
  // ejecutar la consulta utilizando el pool importado, para no crear uno
  nuevo
  const [result] = await pool.query(queries.crearPelicula, [
    name,
    description,
    duration,
    poster,
  ]);

  // la consulta se ejecutó correctamente, entonces enviar la respuesta
  con el resultado
  res.status(201).json({
    message: "Película creada exitosamente",
    result,
  });
} catch (error) {
  console.log("Error en el catch:", error);
  res.status(500).json({
    message: "Error al crear la película: ",
    error,
  });
}
};

const buscarPeliculaPorNombre = async (req, res) => {
  const { name } = req.query;

  // validar que el nombre sea un string no vacío
  if (typeof name !== "string" || name.trim() === "") {
    return res.status(400).json({ message: "El nombre es obligatorio" });
  }

  try {
    // ejecutar la consulta con pool importado
    const [result] = await pool.query(queries.buscarPeliculaPorNombre,
[name]);

```

```

    if (result.length === 0) {
      return res.status(404).json({ message: "No se encontró la película"
});
    }
    res.json(result); // enviar las películas encontradas
  } catch (error) {
    res.status(500).json({ message: "Error al buscar la película", error });
  }
};

const actualizarPelicula = async (req, res) => {
  let { id } = req.params; // se toma el id desde la URL
  id = parseInt(id); // convertir a entero
  let { name, description, duration, poster } = req.body;

  // Validar que el id sea un número válido
  if (!id || isNaN(id)) {
    return res
      .status(400)
      .json({
        message:
          "El id de la película es obligatorio y debe ser un número
válido.",
      });
  }

  // validar que el nombre sea un string no vacío
  if (!name || typeof name !== "string" || name.trim() === "") {
    return res.status(400).json({
      message:
        "El nombre de la película es obligatorio y debe ser un texto
válido.",
    });
  }

  // validar la duración, que sea un entero y mayor a cero
  if (isNaN(duration) || duration <= 0) {
    return res.status(400).json({
      message: "La duración debe ser un número mayor que 0.",
    });
  }

  // validar la descripción, que sea un string no vacío
  if (typeof description !== "string" || description.trim() === "") {
    return res.status(400).json({

```

```

        message: "La descripción es obligatoria y debe ser un texto válido.",
    });
}

try {
    // ejecutar la consulta utilizando el pool importado, para no crear uno
    nuevo
    const [result] = await pool.query(queries.actualizarPelicula, [
        name,
        description,
        duration,
        poster,
        id,
    ]);

    // Verificar si se actualizó alguna película
    if (result.affectedRows === 0) {
        return res.status(404).json({ message: "Película no encontrada" });
    }

    // la consulta se ejecutó correctamente, entonces enviar la respuesta
    con el resultado
    res.status(201).json({
        message: "Película actualizada exitosamente",
        result,
    });
} catch (error) {
    console.log("Error en la actualización:", error);
    res.status(500).json({
        message: "Erro al actualizar la pelicula: ",
        error,
    });
}
};

const eliminarPelicula = async (req, res) => {
    let { id } = req.params; // se toma el id desde la URL
    id = parseInt(id); // convertir a entero

    // Validar que el id sea un número válido
    if (!id || isNaN(id)) {
        return res
            .status(400)
            .json({
                message:

```



```

        "El id de la película es obligatorio y debe ser un número
válido.",
    });
}

try {
    // ejecutar la consulta utilizando el pool importado, para no crear uno
nuevo
    const [result] = await pool.query(queries.eliminarPelicula, [id]);

    // Verificar si se eliminó alguna película
    if (result.affectedRows === 0) {
        return res.status(404).json({ message: "Película no encontrada" });
    }

    // la consulta se ejecutó correctamente, entonces enviar la respuesta
con el resultado
    res.status(201).json({
        message: "Película eliminada exitosamente",
        result,
    });
} catch (error) {
    console.log("Error en la eliminación:", error);
    res.status(500).json({
        message: "Error al eliminar la pelicula: ",
        error,
    });
}
}

const listarPeliculas = async (req, res) => {
    try {
        const [result] = await pool.query(queries.listarPeliculas);
        res.json(result);
    } catch (error) {
        res.status(500).json({ message: "Error al listar las peliculas", error
});
    }
};

module.exports = { crearPelicula, buscarPeliculaPorNombre,
actualizarPelicula, eliminarPelicula, listarPeliculas };

```

usuario.controller.js

```
const pool = require("../config/db");
const queries = require("../database/usuariosQueries");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");

const obtenerUsuarios = async (req, res) => {
  try {
    const [rows] = await pool.query(queries.obtenerUsuarios);
    res.json(rows);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al obtener usuarios", error });
  }
};

const registrarUsuario = async (req, res) => {
  let { name, userName, password } = req.body;

  // validaciones
  if (!name || !userName || !password) {
    return res
      .status(400)
      .json({ mensaje: "Todos los campos son obligatorios" });
  }
  if (password.length < 8) {
    return res
      .status(400)
      .json({ mensaje: "La contraseña debe tener al menos 8 caracteres" });
  }

  try {
    // verificar si el usuario ya existe
    const [userExists] = await pool.query(queries.obtenerUsuarioPorNombre, [
      userName,
    ]);
    if (userExists.length > 0) {
      return res.status(400).json({ mensaje: "El usuario ya existe" });
    }

    // generar hash de la contraseña
    const saltRounds = 10;
    const hashedPassword = await bcrypt.hash(password, saltRounds);

    // registrar usuario
    await pool.query(queries.registrarUsuario, [
```

```

        name,
        userName,
        hashedPassword,
    ]);

    res.status(201).json({ mensaje: "Usuario Registrado Exitosamente" });
} catch (error) {
    console.log("Error en el registro: ", error);
    res.status(500).json({ mensaje: "Error al registrar usuario", error });
}
};

const loginUsuario = async (req, res) => {
    const { userName, password } = req.body;

    // validaciones
    if (!userName || !password) {
        return res
            .status(400)
            .json({ mensaje: "Usuario y contraseña son obligatorios" });
    }

    try {
        // buscar el usuario en la BD
        const [user] = await pool.query(queries.obtenerUsuarioPorNombre, [
            userName,
        ]);

        if (user.length === 0) {
            return res.status(400).json({ mensaje: "Usuario no encontrado" });
        }

        const usuario = user[0];

        // comparar la contraseña ingresada con la encriptada en BD
        const passwordMatch = await bcrypt.compare(password, usuario.password);

        if (!passwordMatch) {
            return res.status(401).json({ mensaje: "Contraseña incorrecta" });
        }

        // generar el JWT
        const token = jwt.sign(
            { userId: usuario.id, userName: usuario.userName, type: usuario.type
},

```

```

    process.env.SECRET_KEY, // clave secreta
    { expiresIn: "1h" } // expira en 1 hora
  );

  res.json({ mensaje: "Inicio de sesión exitoso", token });
} catch (error) {
  console.log("Error en el login: ", error);
  res.status(500).json({ mensaje: "Error al iniciar sesión", error });
}
};

const cambiarRolUsuario = async (req, res) => {
  const { userId } = req.params; // para recibir el id del usuario a cambiar
  const { nuevoRol } = req.body;
  const { userIdAdmin } = req.user; // obtener el id del usuario admin

  // Validar que el rol sea 'cliente' o 'admin'
  if (nuevoRol !== "cliente" && nuevoRol !== "admin") {
    return res
      .status(400)
      .json({ message: "El rol debe ser 'cliente' o 'admin'." });
  }

  try {
    // verificar que el usuario que va a cambiar el rol sea admin
    const [adminCheck] = await pool.query(queries.obtenerTipoDeUsuario, [
      userIdAdmin,
    ]);
    if (adminCheck.length === 0 || adminCheck[0].type !== "admin") {
      return res
        .status(403)
        .json({ message: "No tienes permisos para cambiar roles." });
    }

    // ya con los permisos, cambiar el rol del usuario
    const [result] = await pool.query(queries.cambiarRolDeUsuario, [
      nuevoRol,
      userId,
    ]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ message: "Usuario no encontrado." });
    }

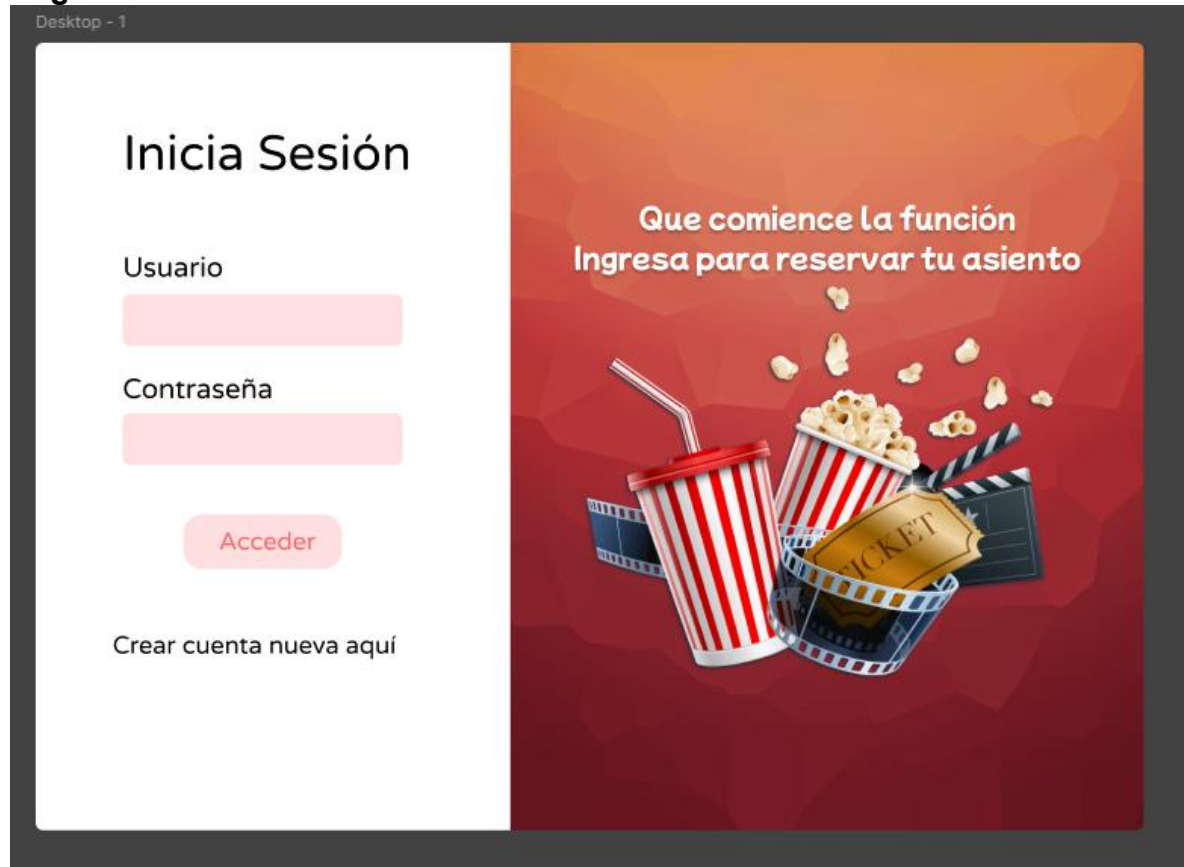
    res.json({ message: `Rol cambiado a ${nuevoRol} exitosamente.` });
  }
};

```

```
    } catch (error) {  
      console.error("Error al cambiar rol:", error);  
      res.status(500).json({ message: "Error al cambiar el rol", error });  
    }  
  };  
  
module.exports = {  
  obtenerUsuarios,  
  registrarUsuario,  
  loginUsuario,  
  cambiarRolUsuario,  
};
```

Mockups

Login



Register



Link del Repositorio

<https://github.com/DanielSajvin/cine-app/tree/develop>